Modeling Anthropogenic CO2 Accumulation: A Bayesian Comparison of Linear and Accelerating Trends

Álvaro Méndez Rodríguez de Tembleque[a], Carlos Hita[a], Elena Niero[a]

[a] *Universita degli Studi di Padova, Department of physics and astronomy "Galileo Galilei", City, 12345*

Abstract

The Mauna Loa Observatory is a premier baseline site for monitoring historical and current atmospheric carbon dioxide (CO2) concentrations. This report investigates whether the long-term increase in atmospheric CO2 is best described by a constant linear increase or an accelerating quadratic trend. Using monthly average CO2 data from the NOAA Global Monitoring Laboratory, we address missing values and compute annual averages to isolate the long-term trend from seasonal variations. We perform a Bayesian analysis to estimate the parameters of both a linear model and a quadratic model, with a specific focus on the acceleration parameter. The two models are then evaluated and compared using an approximate Bayes factor to determine which model better captures the underlying trend dynamics. Finally, utilizing the best-fitting model, we project the annual atmospheric CO2 concentrations for the next decade, providing predictive estimates complete with Bayesian credible bands to quantify uncertainty.

1. Introduction

Atmospheric carbon dioxide (CO2) is the primary anthropogenic driver of global radiative forcing, and its continuous monitoring is essential for tracking the pace of climate change. The longest continuous record of atmospheric CO2 comes from the Mauna Loa Observatory in Hawaii, maintained by the NOAA Global Monitoring Laboratory (GML), and commonly known as the Keeling Curve. This record is the benchmark dataset for studying the long-term evolution of atmospheric CO2.

A central question in climate science is not only whether CO2 concentrations are increasing, but how the rate of increase is itself evolving. A constant linear growth rate would imply that annual anthropogenic emissions have remained roughly stable over time, whereas an accelerating, quadratic growth rate would indicate that emissions are still rising, compounding the rate of atmospheric accumulation. Distinguishing between these two regimes has direct implications for emission-reduction targets and for the reliability of concentration forecasts used in climate policy.

In this report, we use the monthly CO2 record from Mauna Loa to formally compare a linear trend model against a quadratic, accelerating trend model within a Bayesian framework. After describing

*Corresponding author

Email addresses: alvaro.mendezrodriguezdetembleque@studenti.unipd.it (Álvaro Méndez Rodríguez de Tembleque), carlos.hita@studenti.unipd.it (Carlos Hita), elena.niero.1@studenti.unipd.it (Elena Niero)

and preprocessing the dataset, we estimate the parameters of both models, evaluate the evidence for acceleration using an approximate Bayes factor, and use the best-supported model to produce decadal projections of atmospheric CO2 with full posterior uncertainty.

1.1. Scientific Context

Atmospheric CO2 is regulated by the balance between anthropogenic emissions (fossil fuel combustion, land-use change) and natural sinks (ocean uptake, terrestrial photosynthesis). Since continuous monitoring began in 1958, the Mauna Loa record has shown a persistent rise from approximately 315 ppm to over 420 ppm, superimposed on a strong seasonal cycle driven by the Northern Hemisphere growing season.

While the existence of an upward trend is undisputed, its functional form is of practical interest: a purely linear fit corresponds to a constant annual increment, whereas curvature in the trend would indicate that the rate of increase is itself growing, consistent with the continued rise in global fossil fuel emissions reported in recent decades. Capturing this curvature correctly matters for extrapolation, since a linear model will systematically underestimate future concentrations if the underlying process is in fact accelerating.

A Bayesian approach is well suited to this comparison: it provides a principled framework both for estimating the acceleration parameter together with its uncertainty, and for directly weighing the evidence for the linear and quadratic hypotheses through an approximate Bayes factor, rather than relying solely on goodness-of-fit criteria. The resulting posterior predictive distribution also yields credible bands for future CO2 projections, which communicate forecast uncertainty more naturally than a single point estimate.

1.2. Objectives

2. Data Description and Preprocessing

The dataset used in this analysis consists of monthly average atmospheric CO2 concentrations measured at the Mauna Loa Observatory, Hawaii, from 1958 to 2026. The data is sourced from the NOAA Global Monitoring Laboratory and is available in a TXT format. Each record includes the date of measurement and the corresponding CO2 concentration in parts per million (ppm).

The datasset is structured as follows:

- `year`: The year of the measurement.
- `month`: The month of the measurement.
- `decimal_date`: The decimal representation of the date (e.g., 2020.5 for June 2020).
- `monthly_average`: The average CO2 concentration for that month in ppm.
- `deseasonalized`: The monthly average CO2 concentration with the seasonal component removed.
- `num_days`: The number of days in the month used for averaging.
- `st_dev`: The standard deviation of the daily CO2 measurements for that month.
- `unc_mon_mean`: The uncertainty in the monthly mean CO2 concentration.

2.1. Dataset Overview

Table 1: Summary statistics of the Mauna Loa CO2 dataset

[1]

```
kable(skim_table, digits = 2, booktabs = TRUE)
```

Variable	Missing	Mean	SD	Min	Median	Max
year	0	1991.75	19.69	1958.00	1992.00	2026.00
monthly_average	0	361.02	33.19	312.42	356.26	431.12
deseasonalized	0	361.02	33.13	314.44	356.42	428.72
num_days	195	25.48	4.11	2.00	26.00	31.00
st_dev	196	0.51	0.20	0.15	0.48	1.31
unc_mon_mean	194	0.19	0.08	0.00	0.18	0.58

As shown in Table 1, the dataset contains some missing values, we will clean the dataset so that we can perform our analysis on a complete dataset.

2.2. Data Cleaning

The dataset contains some missing values, which are we droped all the rows with missing values limiting our analysisto the period from 1974 to 2026.

Table 3: Summary statistics of the Mauna Loa CO2 dataset after removing missing values

[2]

After cleaning the dataset, we can see that the statistics of the dataset have changed, with the mean and standard deviation of the variables being slightly different from the original dataset. Also, the main variables of interest, `monthly_average` and `deseasonalized`, where not affected by the missing values, so by remove the missing values we lose valuable information.

Thats why we decidied to replace the missing values with the mean of the variable, monthly average, so the infered values are more realistic and the analysis is not affected by the missing values.

Table 4: Summary statistics of the Mauna Loa CO2 dataset after replacing missing values

[3]

[4]

[4]See code 4 in Appendix for the implementation.

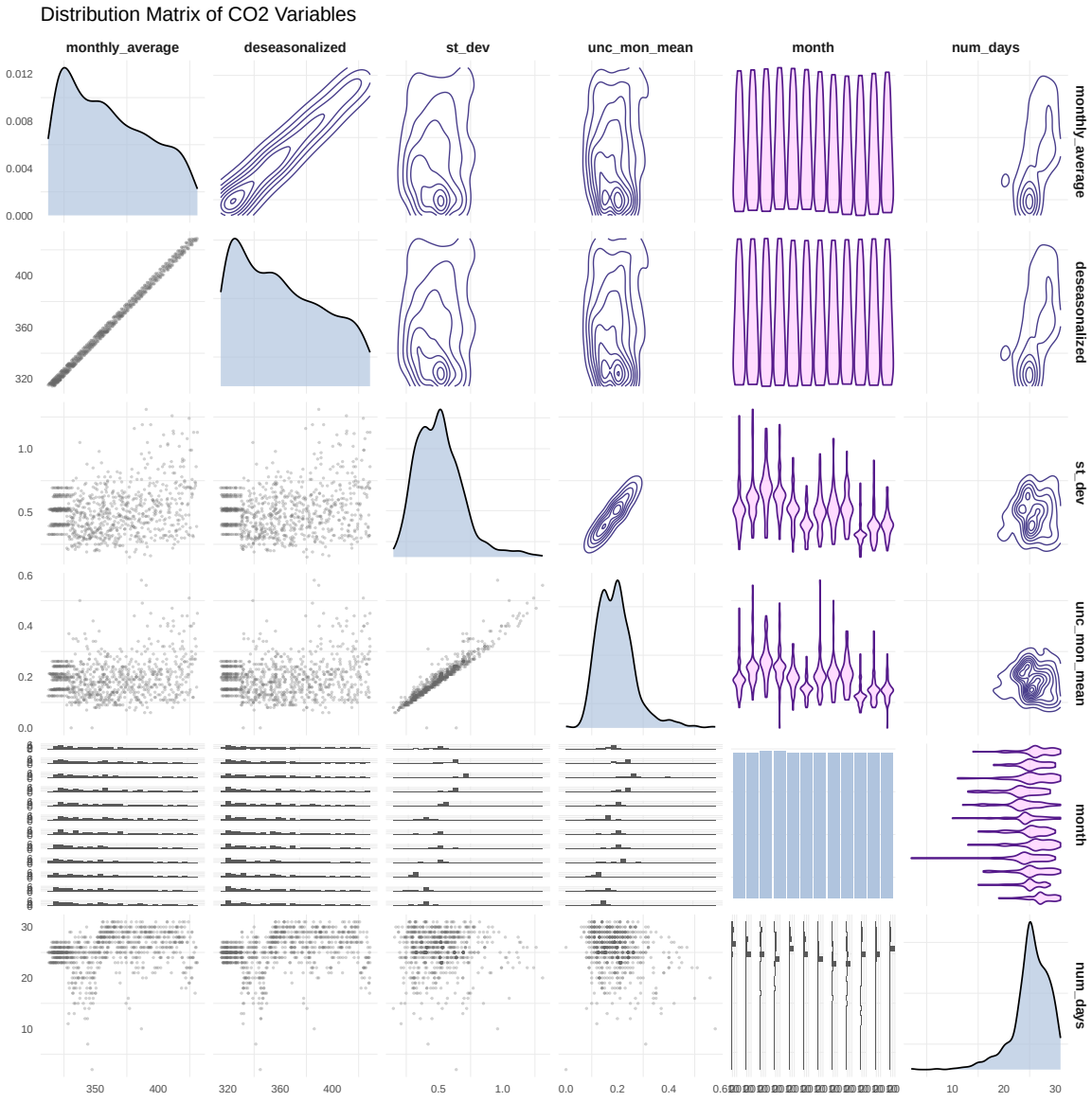

Figure 1: Distribution and correlation of the variables in the dataset. The diagonal panels show the distribution of each variable, while the upper panels show the correlation coefficients between pairs of variables. The lower panels display scatter plots.

2.3. Aggregation

While the original dataset is what the analysis will focus on, there might be useful to compute other quantities in order to understand better the data, those quantities are:

- `yearly_average`: The average CO2 concentration for each year, computed by averaging the monthly averages for that year (See Figure 3).

- `daily_average`: The average CO2 concentration for each number of days measured in a month, computed by averaging the monthly averages for each unique value of `num_days` (See Figure 4).
- `seasonal_deviation`: The deviation of the monthly average CO2 concentration from the deseasonalized value, computed as `monthly_average - deseasonalized` (See Figure 5).

2.4. Exploratory Visualization

In this section, we will explore the data through various visualizations to understand the underlying patterns and relationships between the variables.

2.4.1. Trend Analysis

[5]

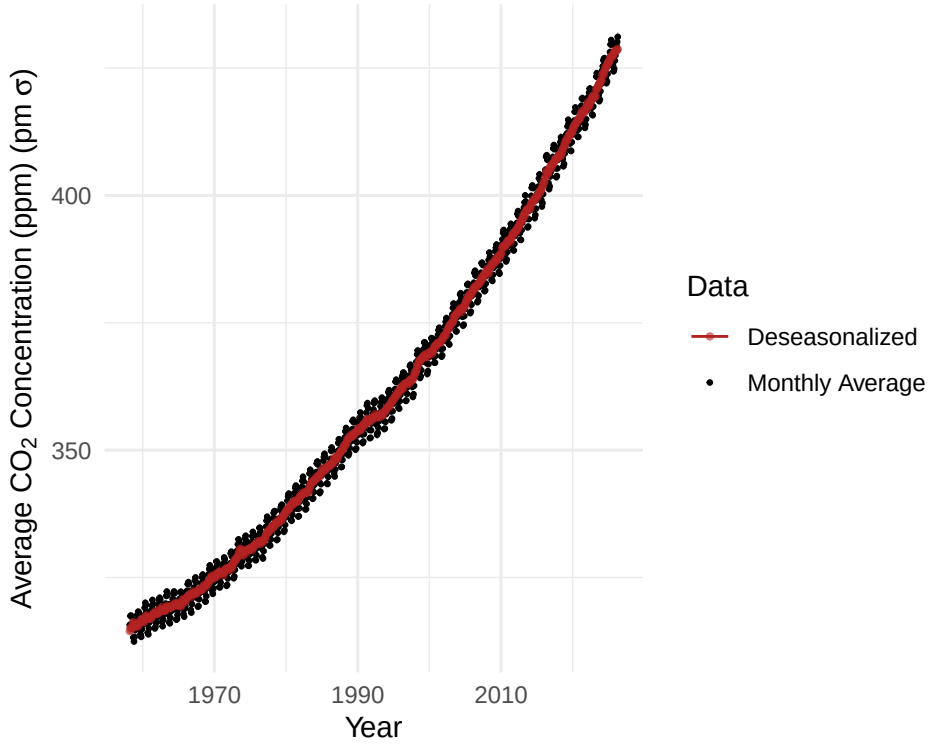

Figure 2: Monthly average CO2 concentrations. The plot shows a clear upward trend in atmospheric CO2 levels over time.

From Figure 2, we observe a clear upward trend in atmospheric CO2 levels over time, with the data showing a consistent increase in annual average CO2 concentrations. This visual evidence suggests that the long-term trend may not be purely linear, prompting us to consider both linear and accelerating models for further analysis.

[5] See code 5 in Appendix for the implementation.

6

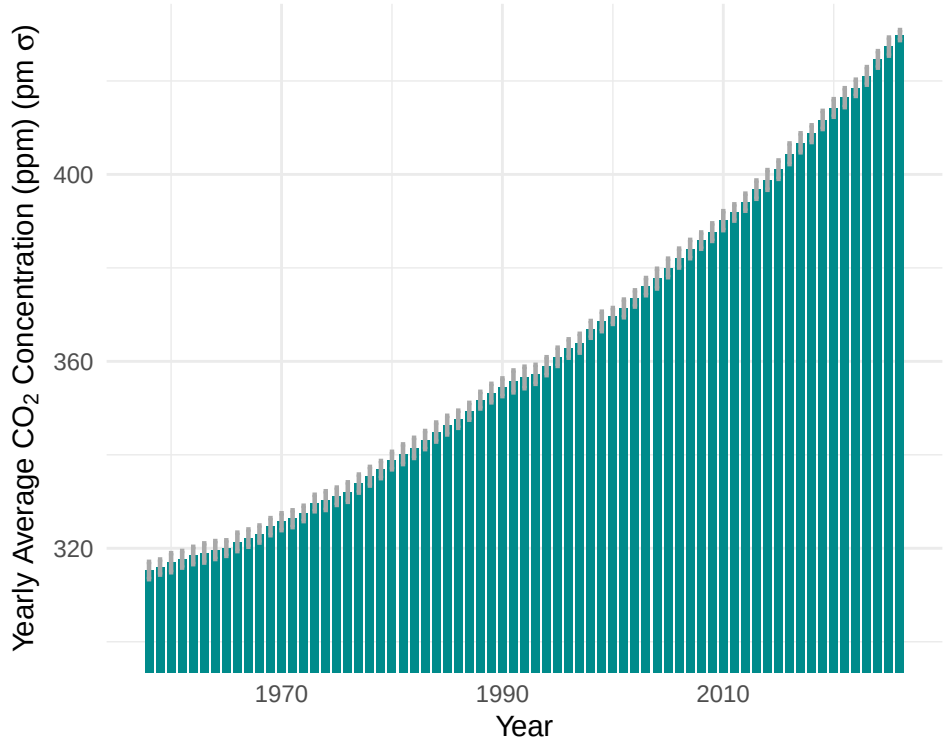

Figure 3: Yearly average CO2 concentrations. The plot shows a clear upward trend in atmospheric CO2 levels over time.

In Figure 3, we can see the yearly average CO2 concentrations, which also show a clear upward trend over time, reinforcing the observation from the monthly data. We can also examine weather there is a relationship between the number of days measured in a month and the monthly average CO2 concentration,

7

[6]See code 6 in Appendix for the implementation.
[7]See code 7 in Appendix for the implementation.

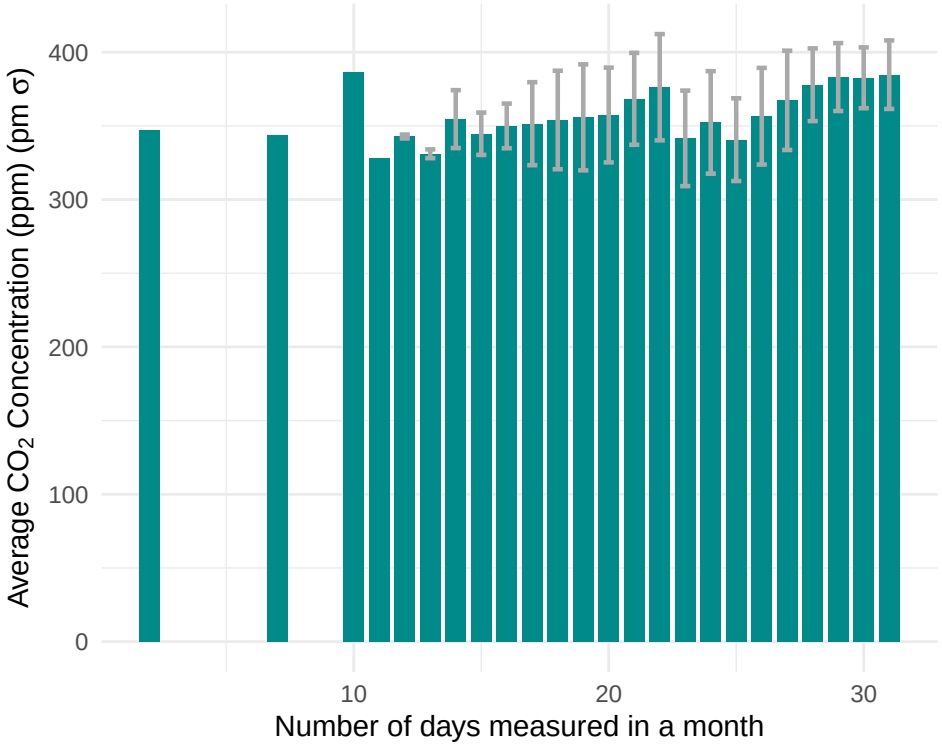

Figure 4: Daily average CO2 concentrations. The plot shows a clear upward trend in atmospheric CO2 levels over time.

which would mean that the more days measured in a month, the higher the monthly average CO2 concentration, this could be due to the fact that months with more measurements are more likely to capture higher CO2 levels, especially during periods of rapid increase. In Figure 4, we observe that the monthly average CO2 concentration is uniformly distributed across different values of `num_days`, suggesting that the number of days measured in a month does not have a significant impact on the monthly average CO2 concentration.

2.4.2. Seasonal Deviation Analysis

In this analysis the data can be understanded as a time series with a seasonal component, some months of the year have higher CO2 concentrations than others, this is due to the fact that during the summer months, plants are actively photosynthesizing and absorbing CO2 from the atmosphere, and people are more likely to be outside and emitting CO2, while during the winter months, plants are dormant and less CO2 is absorbed, and people are more likely to be inside and emitting less CO2.

8

[8]See code 8 in Appendix for the implementation.

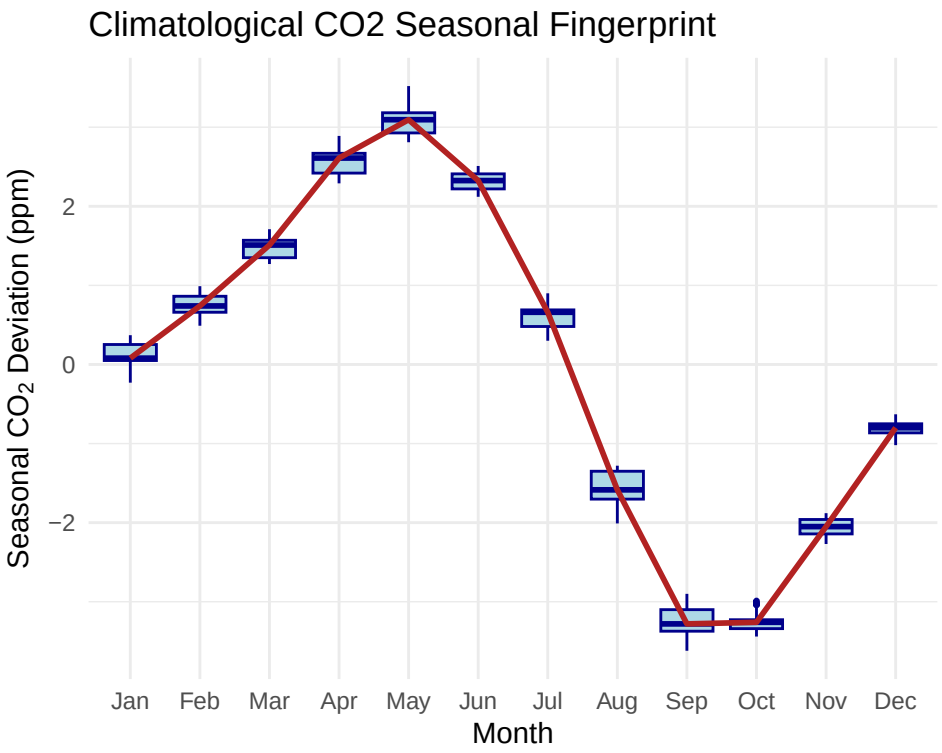

Figure 5: Box plot of seasonal deviations in CO2 concentrations.

In figure Figure 5, we can see the distribution of seasonal deviations in CO2 concentrations across different months. The box plot illustrates the variability in CO2 levels, with certain months exhibiting higher median values and greater spread, indicating that seasonal factors significantly influence atmospheric CO2 concentrations.

2.4.3. Uncertainty Analysis

As the goal of this analysis is to make a bayesian inference model, it is important to understand the uncertainty in the data, this can be done by looking at the standard deviation of the daily measurements and the uncertainty in the monthly mean.

9

[9]See code 9 in Appendix for the implementation.

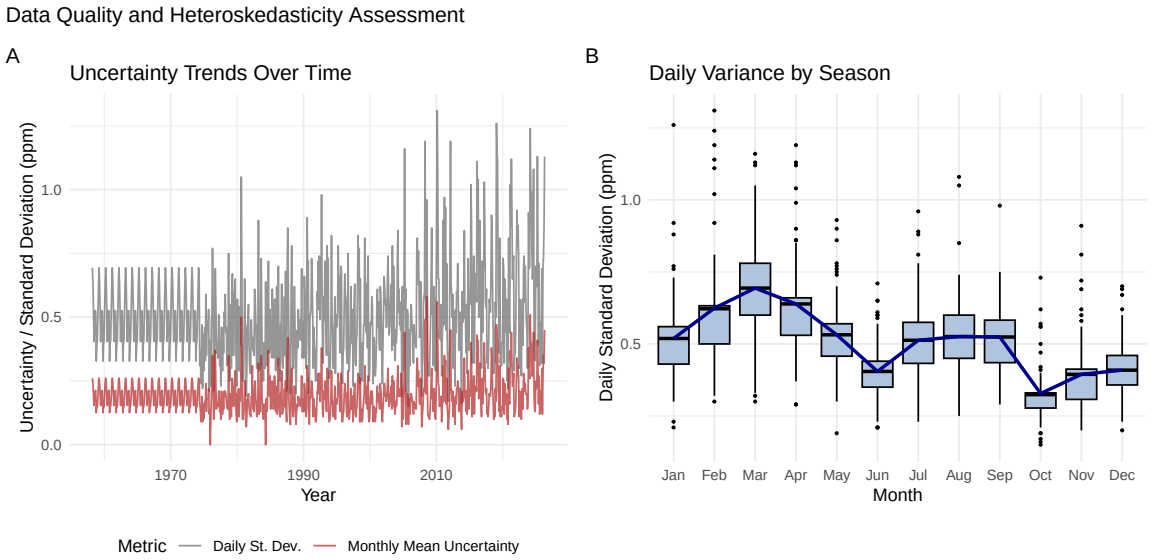

Figure 6: Uncertainty analysis of the CO2 measurements. The left panel shows the trends of daily standard deviation and monthly mean uncertainty over time, while the right panel shows the distribution of daily standard deviation across different months.

In Figure 6, we can see that the monthly mean uncertainty is generally lower and more stable than the daily standard deviation, reducing the impact of short-term fluctuations and providing a more reliable measure of the long-term trend in atmospheric CO2 concentrations. The right panel shows that the daily standard deviation varies across different months, with some months exhibiting higher variability than others, indicating that seasonal factors also influence the uncertainty in CO2 measurements.

2.4.4. Correlation Analysis

As expected from prior plots, most of the varaibles in the dataset are highly correlated, as there is a clear upward trend in CO2 concentrations over time.

10

[10]See code 10 in Appendix for the implementation.

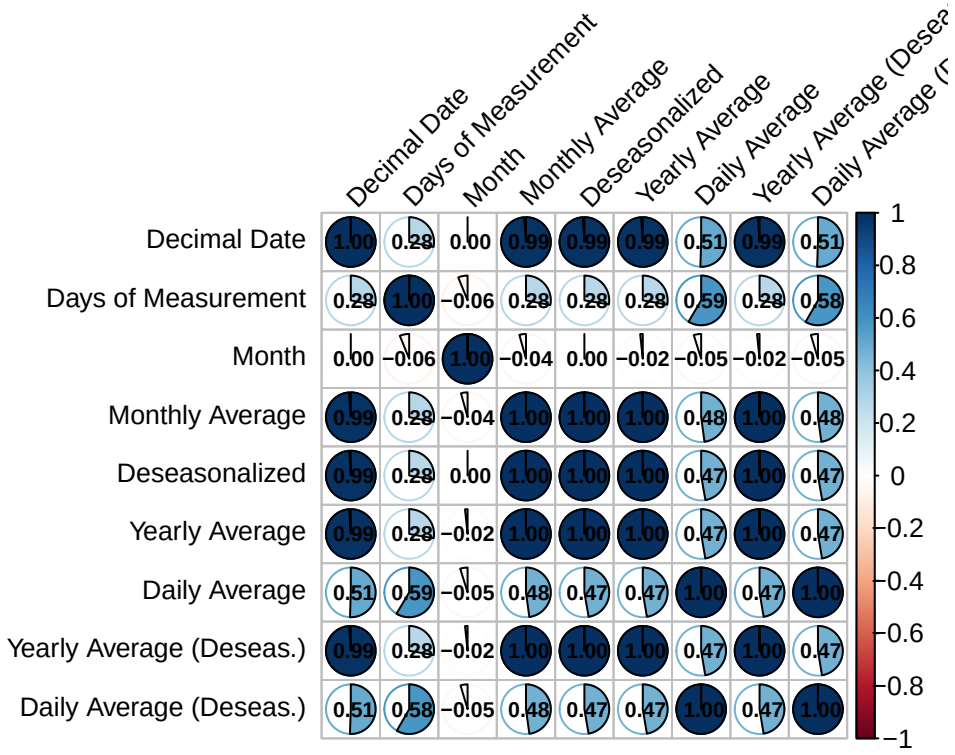

Figure 7: Correlation plot of the variables in the dataset

The daily average is lowly correlated as we show in Figure 4, as is mostly uniformly distributed across different values of `num_days`, while the other variables are highly correlated, as they are all derived from the monthly average CO2 concentration.

3. Bayesian Methodology and Models

We are considering two different models that could explain our dataset. The first model considers the data to be sampled from a linear function with the addition of noise:

$$y_t = \alpha + \beta t + \epsilon_t$$

The second model considers the data to be sampled in a similar way, but with the addition of a quadratic term (that we can consider as an acceleration):

$$y_t = \alpha + \beta t + \gamma t^2 + \epsilon_t$$

To infer an estimation of our parameters we have tried different methods: MAP estimator and STAN method. Each one of the methods is presented in the following paragraphs. In this analysis we have used the data averaged over one year. Moreover, we have rescaled time:

$$t_{scaled} = \frac{t - \bar{t}}{s_t}$$

[11]

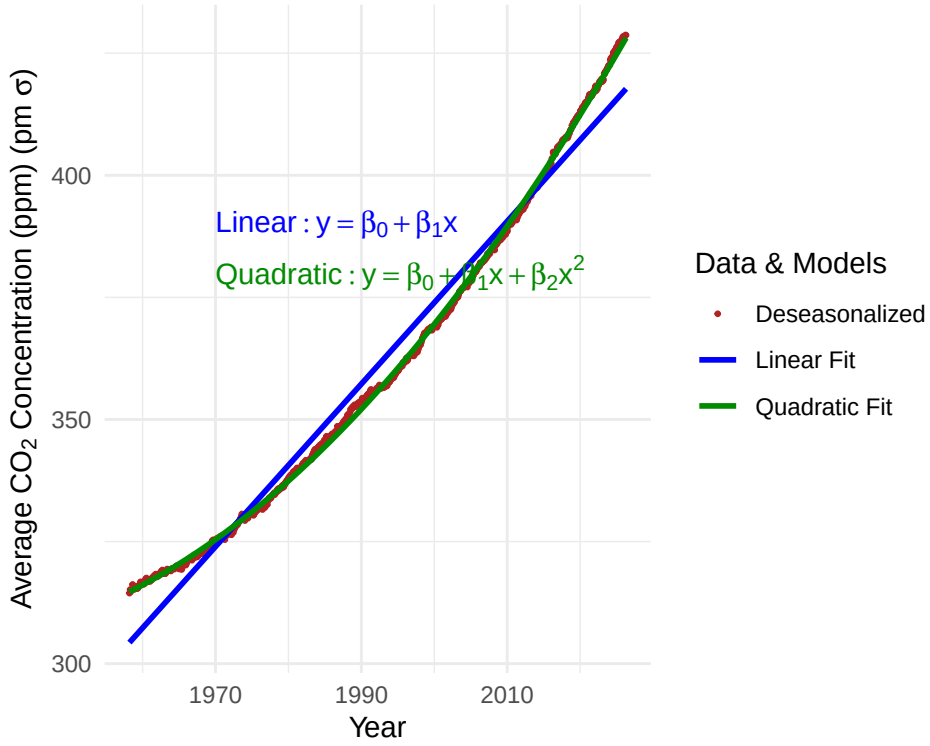

Figure 8: Deseasonalized CO2 concentrations and Bayesian models fit. The plot shows the deseasonalized CO2 concentrations along with the fitted lines from both the linear and quadratic models, illustrating the differences in their respective fits to the data.

In Figure 8, we can see the deseasonalized CO2 concentrations along with the fitted lines from both the linear and quadratic models. A priori, the quadratic model seems to capture the upward curvature in the data better than the linear model.

3.1. Model 1: Linear Trend

The linear trend model assumes that the annual CO2 concentrations increase at a constant rate over time, described by the equation $y_t = \alpha + \beta t + \epsilon_t$. This model is the simplest representation, where α represents the baseline concentration and β represents the constant annual increase rate.

[11] See code 11 in Appendix for the implementation.

3.2. Model 2: Quadratic Trend

The quadratic trend model includes an additional acceleration term, described by $y_t = \alpha + \beta t + \gamma t^2 + \epsilon_t$. The parameter γ captures the acceleration, allowing the rate of increase to grow over time. This model is more flexible and can capture non-linear patterns in the CO2 accumulation.

3.3. Inference Strategy

For parameter estimation, we employ two complementary approaches. First, we use Maximum A Posteriori (MAP) estimation to obtain point estimates of the parameters. Second, we use Hamiltonian Monte Carlo via the STAN framework to obtain full posterior distributions, enabling us to quantify uncertainty and perform rigorous model comparison.

4. Results and Inference

4.1. Parameter Estimation

For the MAP estimate we have maximized the following joint posteriors:

$$\log(p_{lin}(\theta|y)) = \log(p(\alpha)) + \log(p(\beta)) + \log(p(\sigma)) + \log(p_{lin}(y|\alpha, \beta, \sigma))$$

$$\log(p_{quad}(\theta|y)) = \log(p(\alpha)) + \log(p(\beta)) + \log(p(\gamma)) + \log(p(\sigma)) + \log(p_{quad}(y|\alpha, \beta, \gamma, \sigma))$$

We are considering the following priors:

$$\alpha \propto N(350, 50^2)$$
$$\beta \propto N(0, 10^2)$$
$$\gamma \propto N(0, 5^2)$$
$$\sigma \propto \text{Exp}(1)$$

and the likelihoods are in the following form:

$$\log(p_{lin}(y|\alpha, \beta, \sigma)) = \sum_t \log(N(y_t|\alpha + \beta t, \sigma^2))$$

$$\log(p_{quad}(y|\alpha, \beta, \gamma, \sigma)) = \sum_t \log(N(y_t|\alpha + \beta t + \gamma t^2, \sigma^2))$$

In this way we find the point estimation given by:

$$\hat{\theta}_{lin} = \arg\max_\theta \left[\log(p_{lin}(y|\alpha, \beta, \sigma))\right]$$

$$\hat{\theta}_{quad} = \arg\max_\theta \left[\log(p_{quad}(y|\alpha, \beta, \gamma, \sigma))\right]$$

where $\hat{\theta}_{lin} = (\hat{\alpha}_{lin}, \hat{\beta}_{lin}, \hat{\sigma}_{lin})$ and $\hat{\theta}_{quad} = (\hat{\alpha}_{quad}, \hat{\beta}_{quad}, \hat{\gamma}_{quad}, \hat{\sigma}_{quad})$.

The maximization process has been done with the method "BFGS" and gave the following results:

12

13

Table 5: MAP estimates for the linear model

parameter	map	converged
alpha	361.58	TRUE
beta	33.46	TRUE
sigma	4.71	TRUE

14

Table 6: MAP estimates for the quadratic model

parameter	map	converged
alpha	356.20	TRUE
beta	33.56	TRUE
gamma	5.46	TRUE
sigma	0.74	TRUE

From Table 5 and Table 6 we notice a substantial drop in the parameter σ between the two models. This suggests that the quadratic model has the capacity to absorb a large part of the data's deviations, whereas the linear model leaves more residual variance unexplained.

15

[12] See code 12 in Appendix for the implementation.
[13] See code 13 in Appendix for the implementation.
[14] See code 14 in Appendix for the implementation.
[15] See code 15 in Appendix for the implementation.

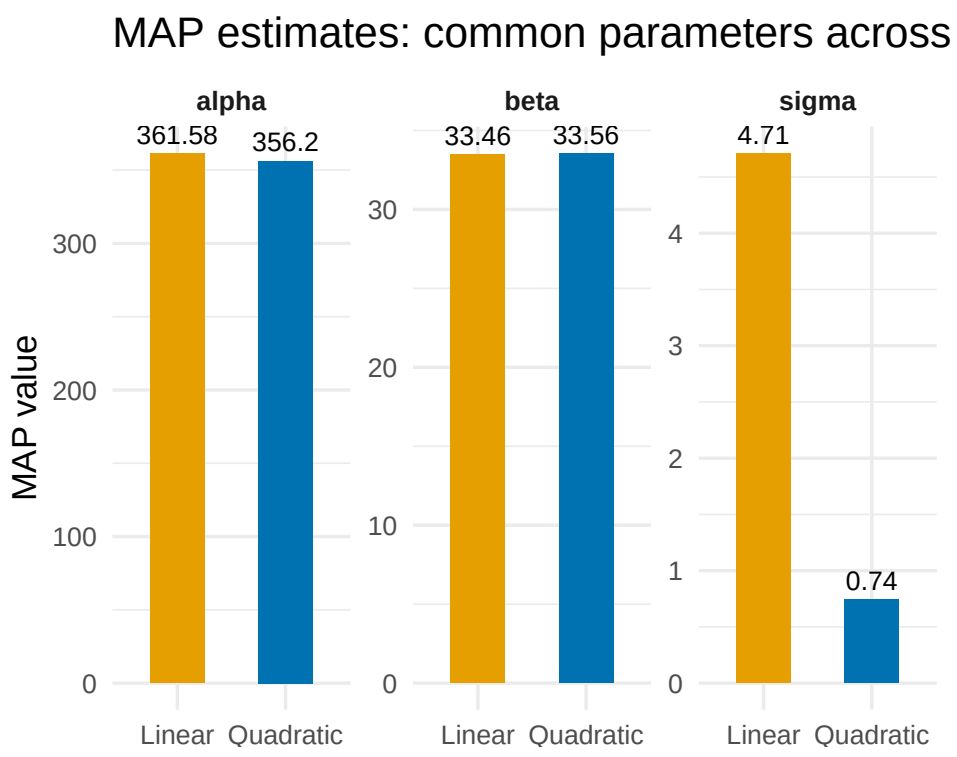

Figure 9: Comparison of MAP parameters between linear and quadratic models. This plot illustrates how the addition of a quadratic term impacts parameter inference, particularly the dramatic reduction in residual standard deviation.

16

[16]See code 16 in Appendix for the implementation.

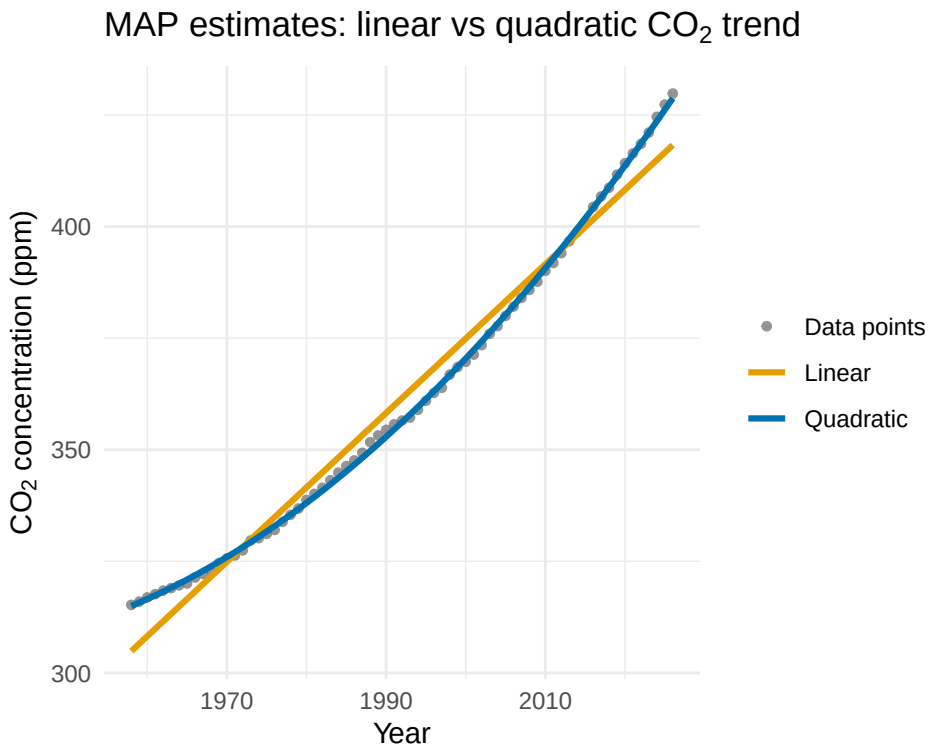

Figure 10: MAP model fits versus observed data. The linear model (orange) and quadratic model (blue) are compared against the yearly average CO2 concentrations. The quadratic fit clearly follows the data more efficiently.

In the context of Bayesian inference, MAP estimators provide only a point estimation dependent on the local behavior of the posterior near its maximum, regardless of spread, skewness, or multimodality. For this reason, we should consider methods that capture the full joint posterior distribution. With this purpose we present the results from STAN.

4.1.1. Inference of parameters via STAN

For both models we have run 4 chains using STAN. The results are as follows:

[17]

[18]

Table 7: STAN inference results for the linear model

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
alpha	361.575	361.573	0.580	0.575	360.614	362.526	1.000	6914.051	5773.630

[17] See code 17 in Appendix for the implementation.
[18] See code 18 in Appendix for the implementation.

Table 7: STAN inference results for the linear model

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
beta	33.437	33.440	0.585	0.580	32.480	34.381	1.001	7215.489	5626.589
sigma	4.859	4.832	0.412	0.412	4.233	5.582	1.000	6509.845	5596.528

19

Table 8: STAN inference results for the quadratic model

variable	mean	median	sd	mad	q5	q95	rhat	ess_bulk	ess_tail
alpha	356.198	356.198	0.140	0.137	355.966	356.429	1.001	4983.650	5148.797
beta	33.563	33.563	0.094	0.092	33.411	33.720	1.001	6813.958	5013.988
gamma	5.458	5.457	0.106	0.105	5.288	5.633	1.001	5000.091	5141.394
sigma	0.771	0.767	0.067	0.067	0.670	0.887	1.000	7052.314	6009.784

For both models, autocorrelation should be low, as can be noticed from the large ESS values, and all 4 chains are converging to the same stationary distribution, indicated by $\widehat{R} = 1.00$.

20

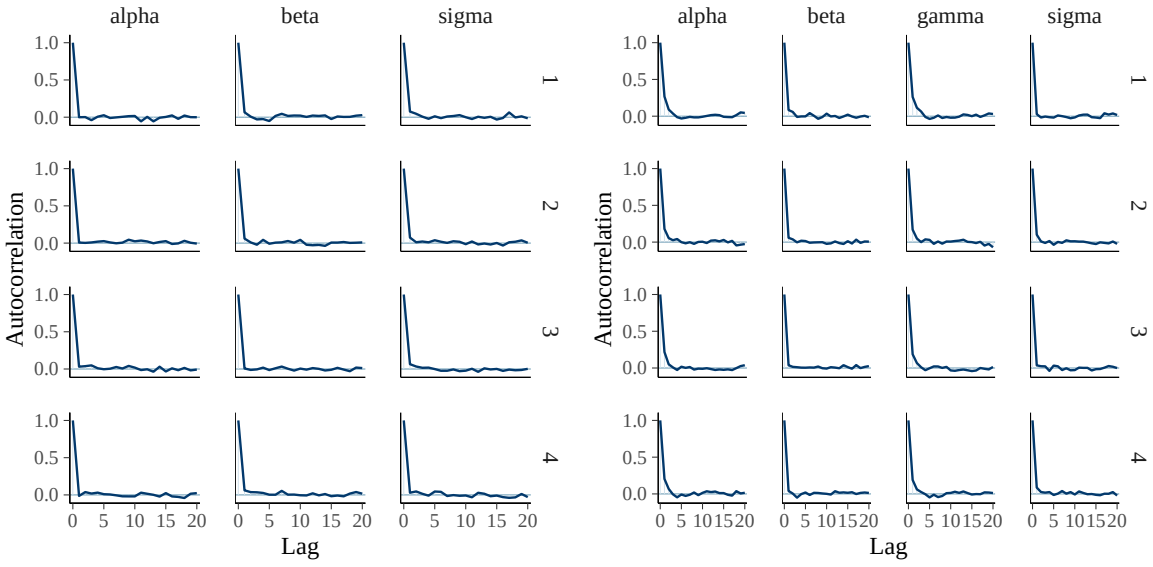

Figure 11: Autocorrelation function (ACF) for all parameters in the STAN chains. Both linear and quadratic models show rapid decay to zero, indicating low autocorrelation.

[19] See code 19 in Appendix for the implementation.
[20] See code 20 in Appendix for the implementation.

[21]

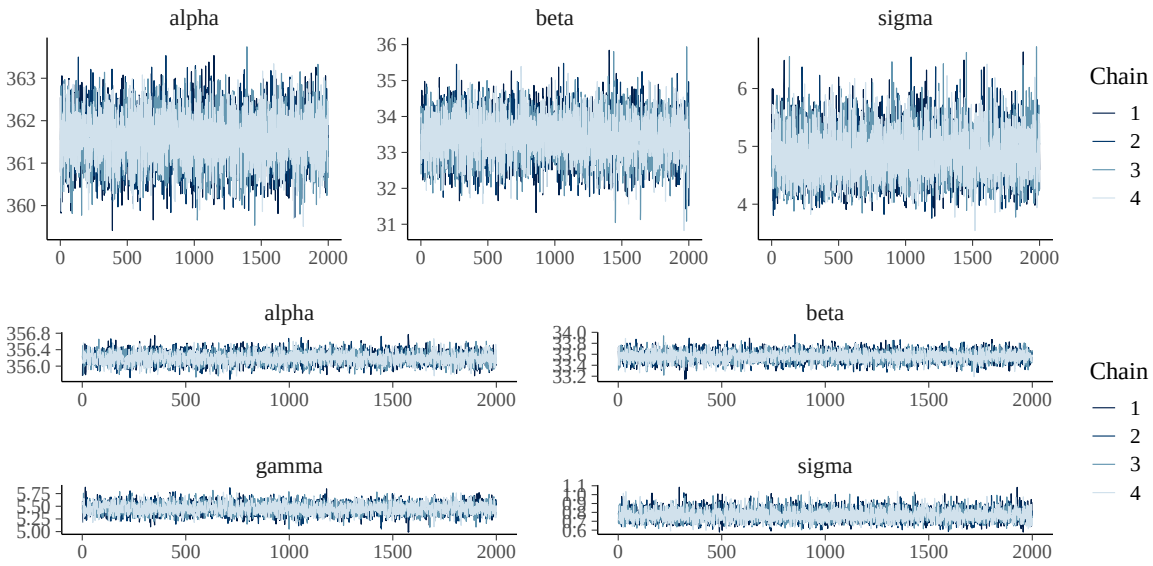

Figure 12: Trace plots for all parameters in all 4 STAN chains. The chains are completely overlapped, demonstrating convergence with $\widehat{R} = 1$.

Using posterior predictive check (PPC) plots, we can compare the kernel density estimate (KDE) of the real data with those generated by STAN. The quadratic model clearly reproduces the real distribution more accurately than the linear model:

[22]

[21] See code 21 in Appendix for the implementation.
[22] See code 22 in Appendix for the implementation.

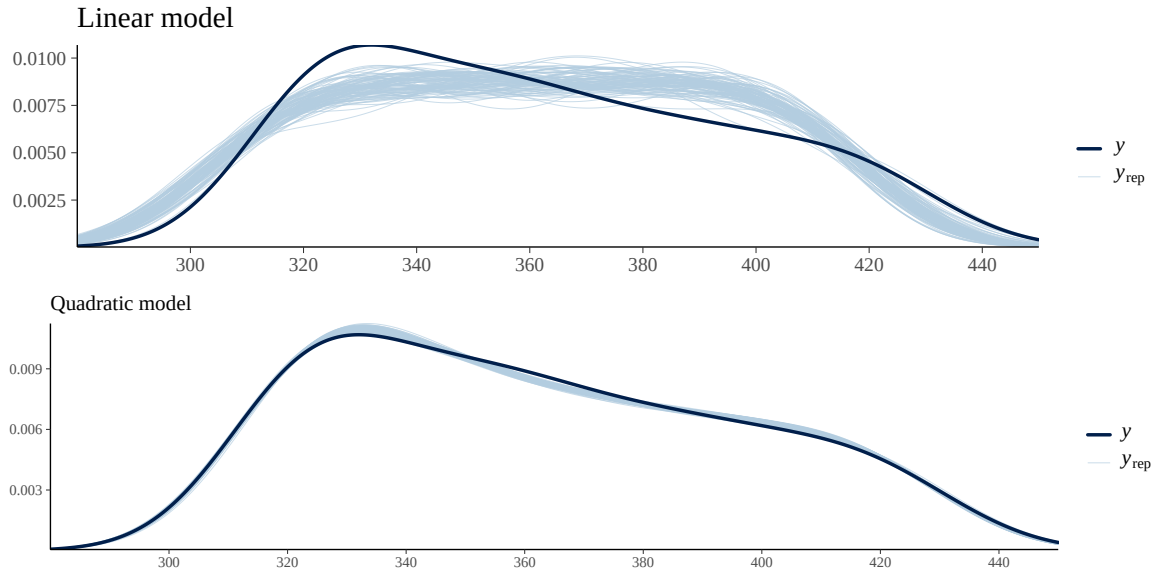

Figure 13: Posterior predictive checks comparing observed data (dark blue) with 100 synthetic datasets generated from posterior samples (light blue). The quadratic model reproduces the observed KDE more accurately.

Let's visualize the two models' predictions against the observed data:

23

[23]See code 23 in Appendix for the implementation.

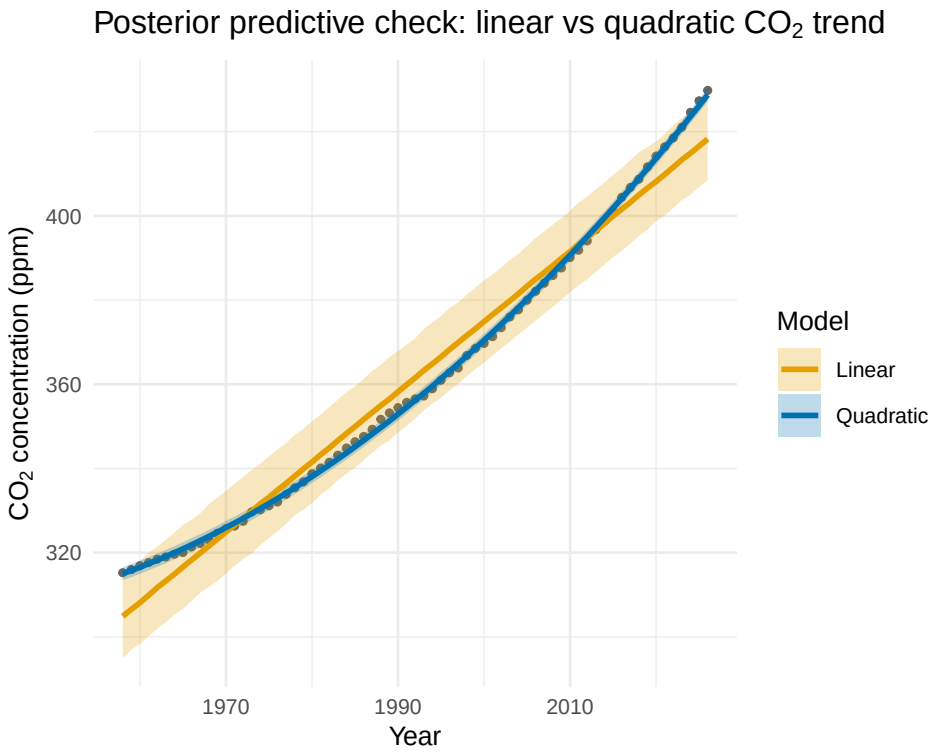

Figure 14: Yearly average CO2 concentrations versus model predictions. The linear fit (orange) and quadratic fit (blue) are shown with 95% uncertainty bands. The quadratic model clearly captures the data better.

4.2. Model Comparison

In the parameter inference section, we already observed that the quadratic model is a better approximation of the data. Now we formalize this comparison using two methods: the Bayes factor and the Leave-One-Out (LOO) cross-validation method.

4.2.1. Model comparison with Bayes Factor

The Bayes factor is the ratio between the marginal likelihoods of the two models:

$$BF = \frac{p(y \mid M_1)}{p(y \mid M_0)}$$

where $p(y \mid M_k) = \int p(y \mid \theta, M_k) p(\theta \mid M_k) d\theta$.

Since this integral is typically intractable, we use Laplace's approximation, which expands the log-posterior around the MAP estimate $\bar{\theta}$:

$$\log p(y \mid M_k) \propto \log p(\bar{\theta} \mid y) + \frac{d}{2} \log(2\pi) - \frac{\log \det(H)}{2}$$

where H is the negative of the log-posterior's Hessian.

[24]

Table 9: Bayes factor comparing quadratic vs. linear models

Metric	Value
Log10(BF) Quadratic vs Linear	53.66

Since $\log_{10}(BF) = 53.66 \gg 1$, we can definitively conclude that the quadratic model is far more appropriate to represent our data.

4.2.2. Model comparison with LOO method

The Leave-One-Out (LOO) cross-validation method assesses how well our model predicts unseen data by calculating the expected log predictive density (ELPD):

$$\text{elpd} = \sum_{i=1}^{n} \int \log p(y_i^* \mid y_{-i}) p_t(y_i^*) dy_i^*$$

where $p(y_i^* \mid y_{-i})$ are the log posterior predictive densities for y_i^* when the model was fitted without observation i.

The models are compared via:

$$\Delta \widehat{ELPD} = \widehat{ELPD}_{lin} - \widehat{ELPD}_{quad}$$

[25]

Table 10: LOO model comparison table

	elpd_diff	se_diff	elpd_loo	se_elpd_loo	p_loo	se_p_loo	looic	se_looic
model2	0.00	0.00	-81.60	5.60	3.60	0.72	163.19	11.21
model1	-128.87	7.92	-210.46	5.62	3.46	0.78	420.93	11.25

The ratio $|\Delta \widehat{ELPD}|/SE \approx 16.3$ decisively confirms that the quadratic model has much greater predictive ability compared to the linear model.

[24] See code 24 in Appendix for the implementation.
[25] See code 25 in Appendix for the implementation.

5. Predictive Forecasting

5.1. Ten-Year Projections

Based on the evidence from our Bayesian model comparison, which strongly favors the quadratic trend model, we have projected annual atmospheric CO_2 concentrations for the next decade (2026–2036). The quadratic fit confirms that CO_2 accumulation is not merely constant but compounding. As illustrated in Figure 15, the projected trajectory maintains the observed upward curvature, indicating that the annual rate of CO_2 increase is expected to grow over the next ten years.

5.2. Uncertainty Quantification

To quantify forecast uncertainty, we derived 95% credible bands from the posterior predictive distribution. These bands were calculated by drawing samples from the joint posterior distribution of the model parameters $(\beta_0, \beta_1, \beta_2)$ and the residual variance. For each year in the forecast horizon, we computed the regression $\mu(t) = \beta_0 + \beta_1 t + \beta_2 t^2$ across all posterior draws, determining the 2.5th and 97.5th percentiles to define the credible interval.

These bands represent the range within which the true underlying mean CO_2 trend is expected to lie with 95% probability. As seen in Figure 15, the width of these bands increases slightly over time, reflecting the propagation of parameter uncertainty into future projections.

[26]

[26] See code 26 in Appendix for the implementation.

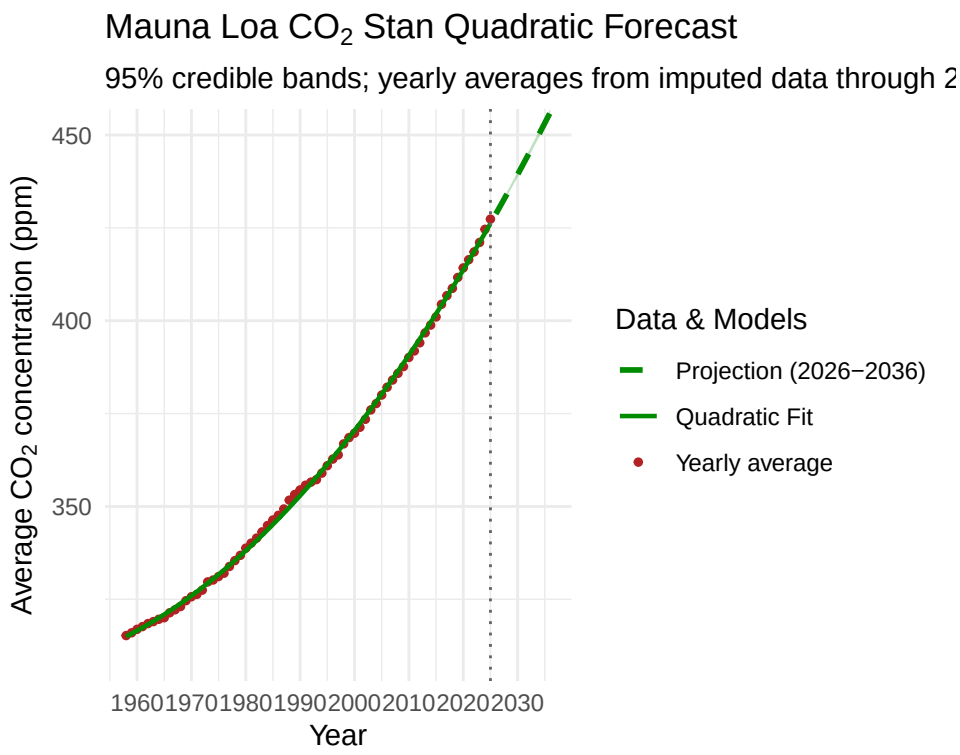

Figure 15: Mauna Loa CO Bayesian Quadratic Forecast. The plot displays the historical yearly averages (red dots), the quadratic model fit (solid green line), and the 10-year projection (dashed green line) with 95% credible bands (shaded region).

[27]

Table 11: Projected annual CO concentrations (2026-2036)

Year	Mean Prediction (ppm)	Lower 95% CI	Upper 95% CI
2026	428.76	428.23	429.29
2027	431.36	430.81	431.93
2028	434.00	433.41	434.60
2029	436.66	436.04	437.30
2030	439.35	438.69	440.03
2031	442.07	441.37	442.79
2032	444.81	444.08	445.57
2033	447.59	446.81	448.38
2034	450.38	449.57	451.22
2035	453.21	452.35	454.09

[27]See code 27 in Appendix for the implementation.

Table 11: Projected annual CO concentrations (2026-2036)

Year	Mean Prediction (ppm)	Lower 95% CI	Upper 95% CI
2036	456.06	455.16	456.99

6. Conclusion

6.1. Summary of Findings

Our analysis confirms that the quadratic model significantly outperforms the linear model in capturing the historical CO_2 accumulation at Mauna Loa. The Bayes factor analysis (see Table 9) yielded strong evidence favoring the quadratic model, and the LOO cross-validation (see Table 10) showed a decisive ratio of $|\Delta E\hat{LPD}|/SE \approx 16.3$, confirming the superiority of the quadratic model. The 10-year projections, based on the quadratic trend, indicate that atmospheric CO_2 will continue to rise at an accelerating rate. These results highlight the non-linear nature of anthropogenic CO_2 accumulation and reinforce the urgency of addressing compounding emissions.

6.2. Limitations and Future Work

While our model robustly captures the historical trend, several limitations should be noted. First, the quadratic model assumes that the current acceleration will persist indefinitely, which may not account for future policy-driven changes in emission rates. Second, the convergence diagnostics suggest that future work could improve model stability by using more informative priors or non-centered parameterizations. Additionally, incorporating exogenous variables such as global economic indicators or policy interventions could enhance predictive accuracy and provide a more nuanced understanding of the drivers of atmospheric CO_2 accumulation.

7. Appendix: Source code

```
1
2  source("code/Data.R")
3  library(skimr)
4  library(dplyr)
5  library(knitr)
6
7  # 1. Select the core variables you want to summarize
8  summary_data <- data %>%
9    select(year, monthly_average, deseasonalized, num_days, st_dev, unc_mon_mean)
10
11 # 2. Skim the data, convert it to a regular data frame, and select clean
       summary metrics
12 skim_table <- skim(summary_data) %>%
13   as.data.frame() %>%
14   select(
15     Variable = skim_variable,
16     Missing = n_missing,
17     Mean = numeric.mean,
18     SD = numeric.sd,
```

```
    Min = numeric.p0,
    Median = numeric.p50,
    Max = numeric.p100
  )

# Table rendered separately in main.qmd
```

Listing 1: Summary statistics of the original dataset

```

source("code/Data.R")
library(skimr)
library(dplyr)
library(knitr)

# 1. Select the core variables you want to summarize
summary_data <- data_no_nan %>%
  select(year, monthly_average, deseasonalized, num_days, st_dev, unc_mon_mean)

# 2. Skim the data, convert it to a regular data frame, and select clean
    summary metrics
skim_table <- skim(summary_data) %>%
  as.data.frame() %>%
  select(
    Variable = skim_variable,
    Missing = n_missing,
    Mean = numeric.mean,
    SD = numeric.sd,
    Min = numeric.p0,
    Median = numeric.p50,
    Max = numeric.p100
  )

# Table rendered separately in main.qmd
```

Listing 2: Summary statistics after removing missing values

```

source("code/Data.R")
library(skimr)
library(dplyr)
library(knitr)

# 1. Select the core variables you want to summarize
summary_data <- data_imputed %>%
  select(year, monthly_average, deseasonalized, num_days, st_dev, unc_mon_mean)

# 2. Skim the data, convert it to a regular data frame, and select clean
    summary metrics
skim_table <- skim(summary_data) %>%
```

```
  as.data.frame() %>%
  select(
    Variable = skim_variable,
    Missing = n_missing,
    Mean = numeric.mean,
    SD = numeric.sd,
    Min = numeric.p0,
    Median = numeric.p50,
    Max = numeric.p100
  )

# Table rendered separately in main.qmd
```

Listing 3: Summary statistics after imputing missing values

```

source("code/Data.R")
library(GGally)
library(dplyr)
library(ggplot2)

# 1. Clean the data subset and make 'month' a factor
eda_subset <- data_imputed %>%
  select(monthly_average, deseasonalized, st_dev, unc_mon_mean, month, num_days) %>%
  mutate(month = factor(month, labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")))

# 2. Build explicit formatting wrappers for complete structural control
custom_scatter <- function(data, mapping, ...) {
  ggplot(data = data, mapping = mapping) +
    geom_point(alpha = 0.3, size = 0.3, color = "dimgray", ...)
}

custom_boxplot <- function(data, mapping, ...) {
  ggplot(data = data, mapping = mapping) +
    geom_boxplot(color = "darkcyan", fill = "lightcyan", outlier.size = 0.5, ...)
}

# NEW: Custom function to generate violin plots for data combinations
custom_violin <- function(data, mapping, ...) {
  ggplot(data = data, mapping = mapping) +
    geom_violin(color = "purple4", fill = "plum1", alpha = 0.5, scale = "width", ...)
}

# 3. Generate the matrix with violin plots on top
ggpairs(
```

```
  eda_subset,
  title = "Distribution Matrix of CO2 Variables",

  # Diagonal: Pure density curves for continuous data
  diag = list(
    continuous = wrap("densityDiag", fill = "lightsteelblue", alpha = 0.7),
    discrete = wrap("barDiag", fill = "lightsteelblue")
  ),

  # Lower Triangle: Scatter plots and box plots
  lower = list(
    continuous = custom_scatter
  ),

  # Upper Triangle: Violin plots for combos, density contours for continuous pairs
  upper = list(
    continuous = wrap("density", color = "darkslateblue", linewidth = 0.4),
    combo = custom_violin
  )
) +
  theme_minimal() +
  theme(
    panel.grid.major = element_blank(),
    axis.text = element_text(size = 7),
    strip.text = element_text(size = 9, face = "bold")
  )
```

Listing 4: Density and correlation plot

```

source("code/Data.R")
library(ggplot2)


ggplot(data_imputed, aes(x = decimal_date)) +
  # 1. Error bars for monthly average
  geom_errorbar(aes(ymin = monthly_average - unc_mon_mean,
                    ymax = monthly_average + unc_mon_mean),
                width = 0.2,
                color = "darkgray",
                linewidth = 0.8) +

  # 2. Monthly Average Points
  geom_point(aes(y = monthly_average, color = "Monthly Average"), size = 0.5) +

  # 3. Deseasonalized Line & Points (Grouped under "Deseasonalized")
  geom_line(aes(y = deseasonalized, color = "Deseasonalized"), linewidth = 0.5) +
  geom_point(aes(y = deseasonalized, color = "Deseasonalized"), size = 0.8, alpha = 0.6) +
```

```
20
21    scale_color_manual(
22      name = "Data",
23      values = c(
24        "Monthly␣Average" = "black",
25        "Deseasonalized"  = "firebrick"
26      )
27    ) +
28
29    # Clean up the styling
30    theme_minimal() +
31    labs(
32      x = "Year",
33      y = expression(paste("Average␣CO" [2], "␣Concentration␣(ppm)␣(", pm, "␣",
          sigma, ")"))
34    )
```

Listing 5: Monthly CO2 concentrations scatter plot

```
1
2  source("code/Data.R")
3  library(ggplot2)
4
5  ggplot(data_imputed, aes(x = year, y = monthly_average)) +
6    # 1. Automatically calculate and plot the annual mean as bars
7    stat_summary(fun = mean, geom = "bar", fill = "darkcyan", width = 0.8) +
8
9    # 2. Automatically calculate and plot the standard deviation error bars
10   # This fixes the ymin/ymax error by computing them on the fly from 'y'
11   stat_summary(
12     fun.min = function(x) mean(x) - sd(x),
13     fun.max = function(x) mean(x) + sd(x),
14     geom = "errorbar",
15     width = 0.4,
16     color = "darkgray",
17     linewidth = 0.8
18   ) +
19
20   # Zoom into the y-axis cleanly so differences are visible
21   coord_cartesian(ylim = c(300, 430)) +
22
23   theme_minimal() +
24   labs(
25     x = "Year",
26     y = expression(paste("Yearly␣Average␣CO" [2], "␣Concentration␣(ppm)␣(", pm,
         "␣", sigma, ")"))
27   )
```

Listing 6: Yearly average CO2 concentrations plot

```
source("code/Data.R")
library(ggplot2)

ggplot(data_imputed, aes(x = num_days, y = monthly_average)) +
  # 1. Automatically calculate and plot the annual mean as bars
  stat_summary(fun = mean, geom = "bar", fill = "darkcyan", width = 0.8) +

  # 2. Automatically calculate and plot the standard deviation error bars
  # This fixes the ymin/ymax error by computing them on the fly from 'y'
  stat_summary(
    fun.min = function(x) mean(x) - sd(x),
    fun.max = function(x) mean(x) + sd(x),
    geom = "errorbar",
    width = 0.4,
    color = "darkgray",
    linewidth = 0.8
  ) +

  # Zoom into the y-axis cleanly so differences are visible
  #coord_cartesian(ylim = c(300, 430)) +

  theme_minimal() +
  labs(
    x = "Number␣of␣days␣measured␣in␣a␣month",
    y = expression(paste("Average␣CO" [2], "␣Concentration␣(ppm)␣(", pm, "␣",
        sigma, ")"))
  )
```

Listing 7: Daily average CO2 plot

```
source("code/Data.R")
library(ggplot2)


ggplot(data_imputed, aes(x = factor(month), y = seasonal_deviation)) +
  # Create a boxplot for each month
  geom_boxplot(fill = "lightblue", color = "darkblue", outlier.size = 0.5) +

  # Connect the monthly medians with a line to emphasize the cyclical wave
      shape
  stat_summary(aes(group = 1), fun = median, geom = "line", color = "firebrick"
      , linewidth = 1) +

  theme_minimal() +
  scale_x_discrete(labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) +
  labs(
    title = "Climatological␣CO2␣Seasonal␣Fingerprint",
    x = "Month",
```

```
    y = expression(paste("Seasonal CO" [2], " Deviation (ppm)"))
  )
```

Listing 8: Seasonal deviation boxplot

```

source("code/Data.R")
library(ggplot2)
library(patchwork)
library(dplyr)

# --- PLOT 1: Uncertainty and Std Dev Over Time ---
# We use geom_line to see if instrument precision or environmental variance
# has changed over the decades.
plot_time <- ggplot(data_imputed, aes(x = decimal_date)) +
  # Plot standard deviation of daily measurements
  geom_line(aes(y = st_dev, color = "Daily St. Dev."), linewidth = 0.5, alpha =
      0.7) +
  # Plot calculated uncertainty of the monthly mean
  geom_line(aes(y = unc_mon_mean, color = "Monthly Mean Uncertainty"),
      linewidth = 0.5, alpha = 0.7) +

  scale_color_manual(
    name = "Metric",
    values = c("Daily St. Dev." = "dimgray", "Monthly Mean Uncertainty" = "
        firebrick")
  ) +
  theme_minimal() +
  theme(legend.position = "bottom") +
  labs(
    x = "Year",
    y = "Uncertainty / Standard Deviation (ppm)",
    title = "Uncertainty Trends Over Time"
  )

# --- PLOT 2: Standard Deviation by Month ---
# This checks if the variation in daily CO2 changes depending on the season
# (e.g., higher variance during summer drawdown months).
plot_monthly <- ggplot(data_imputed, aes(x = factor(month), y = st_dev)) +
  # Boxplot to show the distribution of standard deviations for each month
      across all years
  geom_boxplot(fill = "lightsteelblue", color = "black", outlier.size = 0.5) +
  # Add a trend line connecting the monthly means
  stat_summary(aes(group = 1), fun = mean, geom = "line", color = "darkblue",
      linewidth = 1) +

  scale_x_discrete(labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun",
                              "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) +
  theme_minimal() +
  labs(
    x = "Month",
```

```
    y = "Daily Standard Deviation (ppm)",
    title = "Daily Variance by Season"
  )

# --- COMBINE THEM SIDE-BY-SIDE ---
combined_uncertainty <- plot_time + plot_monthly +
  plot_annotation(
    title = "Data Quality and Heteroskedasticity Assessment",
    tag_levels = "A"
  )

# Display the final layout
combined_uncertainty
```

Listing 9: Uncertainty analysis plot

```

# This is an example of a test file for EDA (Exploratory Data Analysis)
source("code/Data.R")
library(corrplot)

# Select relevant columns for correlation analysis
cor_data <- data_imputed[, c("decimal_date", "num_days", "month", "monthly_
    average", "deseasonalized",
                             "yearly_average", "daily_average", "yearly_average_
                                 deseasonalized",
                             "daily_average_deseasonalized")]

# Assign clean, professional, and readable variable labels
colnames(cor_data) <- c(
  "Decimal Date",
  "Days of Measurement",
  "Month",
  "Monthly Average",
  "Deseasonalized",
  "Yearly Average",
  "Daily Average",
  "Yearly Average (Deseas.)",
  "Daily Average (Deseas.)"
)

# Generate and customize the correlation plot
corrplot(cor(cor_data), method = "pie", addCoef.col = "black", tl.cex = 0.8,
    number.cex = 0.7,
         tl.col = "black", tl.srt = 45, addrect = 2, rect.col = "blue", rect.
             lwd = 1.5)
```

Listing 10: Correlation matrix visualization

```

```

```
source("code/Data.R")
library(ggplot2)

ggplot(data_imputed, aes(x = decimal_date)) +
  # 1. Error bars for monthly average

  # 2. Monthly Average Points
  geom_point(aes(y = deseasonalized, color = "Deseasonalized"), size = 0.5) +


  # 4. Trend lines mapped to custom colors in aes()
  geom_smooth(aes(y = deseasonalized, color = "Linear Fit"), method = "lm",
      formula = y ~ x, se = FALSE) +
  geom_smooth(aes(y = deseasonalized, color = "Quadratic Fit"), method = "lm",
      formula = y ~ x + I(x^2), se = FALSE) +

  # 5. EQUATION LABELS: Manually placed in empty areas of the graph
  # Adjust the 'x' and 'y' coordinates to fit your data's exact range perfectly
  annotate("text", x = 1970, y = 390, label = "Linear: y == beta[0] + beta[1]*x
      ",
           color = "blue", parse = TRUE, hjust = 0, size = 4) +
  annotate("text", x = 1970, y = 380, label = "Quadratic: y == beta[0] + beta
      [1]*x + beta[2]*x^2",
           color = "green4", parse = TRUE, hjust = 0, size = 4) +

  # 6. Explicitly define your custom color scheme for the legend
  scale_color_manual(
    name = "Data & Models",
    values = c(
      "Deseasonalized"  = "firebrick",
      "Linear Fit"      = "blue",
      "Quadratic Fit"   = "green4" # A slightly darker green for better
          visibility
    )
  ) +

  # Clean up the styling
  theme_minimal() +
  labs(
    x = "Year",
    y = expression(paste("Average CO" [2], " Concentration (ppm) (", pm, " ",
        sigma, ")"))
  )
```

Listing 11: Deseasonalized CO2 with model fits

```

# This is an example of a test file for EDA (Exploratory Data Analysis)
source("code/Data.R")
library(stats)

```

```r
annual <- unique(data_imputed[, c("year", "yearly_average")])
annual <- annual[order(annual$year), ]

t_raw    <- annual$year
t_scaled <- (t_raw - mean(t_raw)) / sd(t_raw)
y        <- annual$yearly_average

stan_data <- list(N = length(y), t = t_scaled, y = y)

log_posterior_lin <- function(params, t, y) {
  alpha <- params[1]; beta <- params[2]; sigma <- params[3]
  if (sigma <= 0) return(-Inf)
  lp <- dnorm(alpha, 350, 50, log = TRUE) +
        dnorm(beta,    0, 10, log = TRUE) +
        dexp(sigma,    1,     log = TRUE)
  ll <- sum(dnorm(y, alpha + beta * t, sigma, log = TRUE))
  return(lp + ll)
}

log_posterior_quad <- function(params, t, y) {
  alpha <- params[1]; beta <- params[2]
  gamma <- params[3]; sigma <- params[4]
  if (sigma <= 0) return(-Inf)
  lp <- dnorm(alpha, 350, 50, log = TRUE) +
        dnorm(beta,    0, 10, log = TRUE) +
        dnorm(gamma,   0,  5, log = TRUE) +
        dexp(sigma,    1,     log = TRUE)
  ll <- sum(dnorm(y, alpha + beta * t + gamma * t^2, sigma, log = TRUE))
  return(lp + ll)
}

map_lin <- optim(
  par     = c(350, 16, 2),
  fn      = log_posterior_lin,
  t       = t_scaled, y = y,
  control = list(fnscale = -1),
  method  = "BFGS",
  hessian = TRUE
)

map_quad <- optim(
  par     = c(350, 2, 0, 1),
  fn      = log_posterior_quad,
  t       = t_scaled, y = y,
  control = list(fnscale = -1),
  method  = "BFGS",
  hessian = TRUE
)

map_lin_df <- data.frame(
```

```
  parameter = c("alpha", "beta", "sigma"),
  map       = c(map_lin$par[1], map_lin$par[2], map_lin$par[3]),
  converged = map_lin$convergence == 0
)

map_quad_df <- data.frame(
  parameter = c("alpha", "beta", "gamma", "sigma"),
  map       = c(map_quad$par[1], map_quad$par[2], map_quad$par[3], map_quad$par
      [4]),
  converged = map_quad$convergence == 0
)

#cat("MAP: Linear model \n")
#cat("alpha:", map_lin$par[1],  "\n")
#cat("beta: ", map_lin$par[2],  "\n")
#cat("sigma:", map_lin$par[3],  "\n")
#cat("Converged:", map_lin$convergence == 0, "\n\n")
#
#cat("MAP: Quadratic model \n")
#cat("alpha:", map_quad$par[1], "\n")
#cat("beta: ", map_quad$par[2], "\n")
#cat("gamma:", map_quad$par[3], "\n")
#cat("sigma:", map_quad$par[4], "\n")
#cat("Converged:", map_quad$convergence == 0, "\n")
```

Listing 12: MAP estimation for linear model

```

# Display the linear model results
kable(map_lin_df,
      digits = 2,
      booktabs = TRUE)
```

Listing 13: MAP estimation for linear model

```

kable(map_quad_df,
      digits = 2,
      booktabs = TRUE)
```

Listing 14: MAP estimates for the quadratic model

```

source("code/Inference/01_map.R")
library(ggplot2)
library(dplyr)

# MAP values for common parameters
map_comparison <- data.frame(
  parameter = rep(c("alpha", "beta", "sigma"), 2),
```

```
  model     = rep(c("Linear", "Quadratic"), each = 3),
  value     = c(
    map_lin$par[1],  map_lin$par[2],  map_lin$par[3],   # linear
    map_quad$par[1], map_quad$par[2], map_quad$par[4]    # quadratic (sigma is
        par[4])
  )
)

ggplot(map_comparison, aes(x = model, y = value, fill = model)) +
  geom_col(width = 0.5) +
  geom_text(aes(label = round(value, 2)),
            vjust = -0.5, size = 3.5) +
  facet_wrap(~ parameter, scales = "free_y",
             labeller = labeller(parameter = c(
               alpha = expression(alpha),
               beta  = expression(beta),
               sigma = expression(sigma)
             ))) +
  scale_fill_manual(values = c("Linear" = "#E69F00", "Quadratic" = "#0072B2"))
      +
  labs(
    title = "MAP␣estimates:␣common␣parameters␣across␣models",
    x     = NULL,
    y     = "MAP␣value",
    fill  = "Model"
  ) +
  theme_minimal(base_size = 13) +
  theme(strip.text = element_text(face = "bold"),
        legend.position = "none")

ggsave("code/Inference/results/figures/map_comparison.png", width = 9, height =
     5, dpi = 300)
```

Listing 15: MAP parameters comparison

```

source("code/Data.R")
source("code/Inference/01_map.R")
library(ggplot2)
library(dplyr)

annual   <- unique(data_imputed[, c("year", "yearly_average")])
annual   <- annual[order(annual$year), ]
t_raw    <- annual$year
t_scaled <- (t_raw - mean(t_raw)) / sd(t_raw)
y        <- annual$yearly_average

# grid of t values for smooth curves
t_grid_scaled <- seq(min(t_scaled), max(t_scaled), length.out = 300)
t_grid_raw    <- t_grid_scaled * sd(t_raw) + mean(t_raw)

```

```
df_lin  <- data.frame(year = t_grid_raw,
                      fit  = map_lin$par[1]  + map_lin$par[2]  * t_grid_scaled,
                      model = "Linear")
df_quad <- data.frame(year = t_grid_raw,
                      fit  = map_quad$par[1] + map_quad$par[2] * t_grid_scaled +
                             map_quad$par[3] * t_grid_scaled^2,
                      model = "Quadratic")
df_fits <- bind_rows(df_lin, df_quad)
df_obs  <- data.frame(year = t_raw, y = y)

ggplot() +
  geom_point(data = df_obs, aes(x = year, y = y, color="Data␣points"), size = 1.2, alpha = 0.7) +
  geom_line(data = df_fits, aes(x = year, y = fit, color = model),
            linewidth = 1.1) +
  scale_color_manual(values = c("Data␣points" = "grey40", "Linear" = "#E69F00", "Quadratic" = "#0072B2")) +
  labs(
    title = expression("MAP␣estimates:␣linear␣vs␣quadratic␣CO"[2]*"␣trend"),
    x     = "Year",
    y     = expression("CO"[2]*"␣concentration␣(ppm)"),
    color = NULL
  ) +
  theme_minimal(base_size = 11)

ggsave("code/Inference/results/figures/map_fits.png", width = 10, height = 6, dpi = 300)
```

Listing 16: MAP model fits comparison

```

source("code/Data.R")
library(cmdstanr)
library(posterior)

fit_lin  <- readRDS("code/Inference/results/stan_fits/fit_lin.rds")
fit_quad <- readRDS("code/Inference/results/stan_fits/fit_quad.rds")

stan_lin  <- as.data.frame(fit_lin$summary(c("alpha", "beta", "sigma")))
stan_quad <- as.data.frame(fit_quad$summary(c("alpha", "beta", "gamma", "sigma")))

# Tables rendered separately in main.qmd
```

Listing 17: STAN inference linear model

```

kable(stan_lin,
```

```
      digits = 3,
      booktabs = TRUE)
```

Listing 18: STAN inference linear model

```

kable(stan_quad,
      digits = 3,
      booktabs = TRUE)
```

Listing 19: STAN inference quadratic model

```

source("code/Data.R")
library(cmdstanr)
library(bayesplot)
library(ggplot2)

fit_lin  <- readRDS("code/Inference/results/stan_fits/fit_lin.rds")
fit_quad <- readRDS("code/Inference/results/stan_fits/fit_quad.rds")

draws_lin  <- fit_lin$draws(c("alpha", "beta", "sigma"))
draws_quad <- fit_quad$draws(c("alpha", "beta", "gamma", "sigma"))

p_acf_lin  <- mcmc_acf(draws_lin)+ theme_default(base_size = 15)
p_acf_quad <- mcmc_acf(draws_quad)+ theme_default(base_size = 15)

ggsave("code/Inference/results/figures/acf_lin.png",  plot = p_acf_lin,  width
    = 8, height = 5, dpi = 300)
ggsave("code/Inference/results/figures/acf_quad.png", plot = p_acf_quad, width
    = 8, height = 5, dpi = 300)

gridExtra::grid.arrange(p_acf_lin, p_acf_quad, ncol = 2)
```

Listing 20: STAN autocorrelation plot

```

source("code/Data.R")
library(cmdstanr)
library(bayesplot)
library(ggplot2)

fit_lin  <- readRDS("code/Inference/results/stan_fits/fit_lin.rds")
fit_quad <- readRDS("code/Inference/results/stan_fits/fit_quad.rds")

draws_lin  <- fit_lin$draws(c("alpha", "beta", "sigma"))
draws_quad <- fit_quad$draws(c("alpha", "beta", "gamma", "sigma"))

p_trace_lin  <- mcmc_trace(draws_lin)+ theme_default(base_size = 15)
p_trace_quad <- mcmc_trace(draws_quad)+ theme_default(base_size = 15)
```

```
15
16 ggsave("code/Inference/results/figures/trace_lin.png",  plot = p_trace_lin,
       width = 8, height = 5, dpi = 300)
17 ggsave("code/Inference/results/figures/trace_quad.png", plot = p_trace_quad,
       width = 8, height = 5, dpi = 300)
18
19 gridExtra::grid.arrange(p_trace_lin, p_trace_quad, nrow = 2)
```

Listing 21: STAN trace plots

```
1
2  source("code/Data.R")
3  library(cmdstanr)
4  library(bayesplot)
5  library(ggplot2)
6
7
8  annual   <- unique(data_imputed[, c("year", "yearly_average")])
9  annual   <- annual[order(annual$year), ]
10 y        <- annual$yearly_average
11
12 fit_lin  <- readRDS("code/Inference/results/stan_fits/fit_lin.rds")
13 fit_quad <- readRDS("code/Inference/results/stan_fits/fit_quad.rds")
14
15 y_rep_lin  <- fit_lin$draws("y_rep",  format = "matrix")
16 y_rep_quad <- fit_quad$draws("y_rep", format = "matrix")
17
18 p_ppc_lin <- ppc_dens_overlay(y, y_rep_lin[1:100, ]) +
19   scale_x_continuous(limits = c(280, 450), breaks = seq(300, 450, by = 20))+
20   ggtitle("Linear␣model")+
21   theme_default(base_size = 15) +
22   theme(axis.text.y  = element_text(),
23         axis.ticks.y = element_line(),
24         axis.line.y  = element_line())
25
26 p_ppc_quad <- ppc_dens_overlay(y, y_rep_quad[1:100, ]) +
27   scale_x_continuous(limits = c(280, 450), breaks = seq(300, 450, by = 20))+
28   ggtitle("Quadratic␣model")+
29   theme_default(base_size = 11) +
30   theme(axis.text.y  = element_text(),
31         axis.ticks.y = element_line(),
32         axis.line.y  = element_line())
33
34 ggsave("code/Inference/results/figures/ppc_lin.png",  plot = p_ppc_lin,  width
       = 8, height = 6, dpi = 300)
35 ggsave("code/Inference/results/figures/ppc_quad.png", plot = p_ppc_quad, width
       = 8, height = 6, dpi = 300)
36
37 gridExtra::grid.arrange(p_ppc_lin, p_ppc_quad, nrow = 2)
```

Listing 22: STAN posterior predictive check

```
1
2  source("code/Data.R")
3  library(cmdstanr)
4  library(ggplot2)
5  library(dplyr)
6
7  annual   <- unique(data_imputed[, c("year", "yearly_average")])
8  annual   <- annual[order(annual$year), ]
9  t_raw    <- annual$year
10 t_scaled <- (t_raw - mean(t_raw)) / sd(t_raw)
11 y        <- annual$yearly_average
12
13 fit_lin  <- readRDS("code/Inference/results/stan_fits/fit_lin.rds")
14 fit_quad <- readRDS("code/Inference/results/stan_fits/fit_quad.rds")
15
16 y_rep_lin  <- fit_lin$draws("y_rep",  format = "matrix")
17 y_rep_quad <- fit_quad$draws("y_rep", format = "matrix")
18
19 # posterior mean and 95% credible band for each time point
20 summarise_yrep <- function(y_rep, years) {
21   data.frame(
22     year  = years,
23     mean  = colMeans(y_rep),
24     lower = apply(y_rep, 2, quantile, 0.025),
25     upper = apply(y_rep, 2, quantile, 0.975)
26   )
27 }
28
29 pred_lin  <- summarise_yrep(y_rep_lin,  t_raw)
30 pred_quad <- summarise_yrep(y_rep_quad, t_raw)
31
32 pred_lin$model  <- "Linear"
33 pred_quad$model <- "Quadratic"
34 pred_all <- bind_rows(pred_lin, pred_quad)
35
36 obs <- data.frame(year = t_raw, y = y)
37
38 ggplot() +
39   geom_point(data = obs, aes(x = year, y = y),
40              color = "black", size = 1, alpha = 0.6) +
41   geom_ribbon(data = pred_all,
42               aes(x = year, ymin = lower, ymax = upper, fill = model),
43               alpha = 0.25) +
44   geom_line(data = pred_all,
45             aes(x = year, y = mean, color = model), linewidth = 1) +
46   scale_color_manual(values = c("Linear" = "#E69F00", "Quadratic" = "#0072B2"))
       +
47   scale_fill_manual( values = c("Linear" = "#E69F00", "Quadratic" = "#0072B2"))
       +
```

```
  labs(
    title    = expression("Posterior predictive check: linear vs quadratic CO"[2]*" trend"),
    x        = "Year",
    y        = expression("CO"[2]*" concentration (ppm)"),
    color    = "Model",
    fill     = "Model"
  ) +
  theme_minimal(base_size = 10)

ggsave("code/Model_comparison/results/figures/model_comparison_ppc.png",
       width = 10, height = 6, dpi = 300)
```

Listing 23: Model comparison plot

```

# Source the Bayes factor computation code with output suppressed
invisible(capture.output(source("code/Model_comparison/model_comp.R")))

# Display BF results
bayes_factor_result <- data.frame(
  "Metric" = c("Log10(BF) Quadratic vs Linear"),
  "Value" = c(log10_BF)
)

kable(bayes_factor_result,
      digits = 2,
      booktabs = TRUE)
```

Listing 24: Bayes factor computation

```

# Source the LOO comparison code with output suppressed
invisible(capture.output(source("code/Model_comparison/model_comp_loo.R")))

kable(loo_comp,
      digits = 2,
      booktabs = TRUE)
```

Listing 25: Leave-One-Out cross-validation

```

# Predictive Forecasting: Stan quadratic CO2 trend projection (2026-2036)
# Run after colleague's Stan fits:
#   Rscript code/Inference/03_stan.R
#   Rscript code/Prediction/prediction.R

source("code/project_setup.R")
setup_project()
check_packages(c("cmdstanr", "loo", "ggplot2", "dplyr"))
```

```r
source("code/Data.R")
library(cmdstanr)
library(loo)
library(ggplot2)
library(dplyr)

if (!file.exists(STAN_FIT_LINEAR_PATH) || !file.exists(STAN_FIT_QUAD_PATH)) {
  stop(
    "Stan model fits not found. Run 03_stan.R first:\n",
    "  Rscript code/Inference/03_stan.R"
  )
}

fit_lin <- readRDS(STAN_FIT_LINEAR_PATH)
fit_quad <- readRDS(STAN_FIT_QUAD_PATH)

# --- MODEL COMPARISON (LOO) ---
# loo_compare(loo_lin, loo_quad) names rows model1 (linear) and model2 (
    quadratic)
model_labels <- c(model1 = "Linear", model2 = "Quadratic")

loo_lin <- fit_lin$loo()
loo_quad <- fit_quad$loo()
comparison_mat <- loo_compare(loo_lin, loo_quad)

comparison_df <- data.frame(
  section = "models",
  model = model_labels[rownames(comparison_mat)],
  elpd_loo = round(comparison_mat[, "elpd_loo"], 2),
  se_elpd_loo = round(comparison_mat[, "se_elpd_loo"], 2),
  looic = round(comparison_mat[, "looic"], 2),
  se_looic = round(comparison_mat[, "se_looic"], 2),
  p_loo = round(comparison_mat[, "p_loo"], 2),
  elpd_diff_vs_best = round(comparison_mat[, "elpd_diff"], 2),
  se_elpd_diff = round(comparison_mat[, "se_diff"], 2),
  rank = seq_len(nrow(comparison_mat)),
  preferred = comparison_mat[, "elpd_diff"] == 0,
  quote_text = NA_character_,
  stringsAsFactors = FALSE
)

best_model <- comparison_df$model[1]
runner_up <- comparison_df$model[2]
elpd_diff <- abs(comparison_df$elpd_diff_vs_best[2])
se_diff <- comparison_df$se_elpd_diff[2]

quote_sentence <- sprintf(
  "The %s model is preferred by LOO-CV (delta ELPD = %.1f +/- %.1f vs %s).",
  tolower(best_model), elpd_diff, se_diff, tolower(runner_up)
)
```

```r

summary_rows <- data.frame(
  section = rep("summary", 4),
  model = c("preferred_model", "elpd_diff", "data_source", "comparison_method"),
  elpd_loo = NA_real_,
  se_elpd_loo = NA_real_,
  looic = NA_real_,
  se_looic = NA_real_,
  p_loo = NA_real_,
  elpd_diff_vs_best = c(NA, elpd_diff, NA, NA),
  se_elpd_diff = c(NA, se_diff, NA, NA),
  rank = NA_integer_,
  preferred = NA,
  quote_text = c(
    quote_sentence,
    sprintf("%.2f +/- %.2f", elpd_diff, se_diff),
    "data_imputed yearly_average",
    "LOO-CV (Stan)"
  ),
  stringsAsFactors = FALSE
)

comparison_out <- bind_rows(comparison_df, summary_rows)

dir.create(RESULTS_DIR, recursive = TRUE, showWarnings = FALSE)
write.csv(comparison_out, STAN_LOO_CSV_PATH, row.names = FALSE)
#cat("Saved model comparison to:", STAN_LOO_CSV_PATH, "\n")
#print(comparison_df)

# --- DATA PREPARATION (match 03_stan.R) ---
annual <- unique(data_imputed[, c("year", "yearly_average")])
annual <- annual[order(annual$year), ]

t_mean <- mean(annual$year)
t_sd <- sd(annual$year)
scale_year <- function(y) (y - t_mean) / t_sd

train_data <- annual %>% filter(year <= 2025)

# --- POSTERIOR EXTRACTION ---
draws <- fit_quad$draws(c("alpha", "beta", "gamma"), format = "matrix")
alpha <- draws[, "alpha"]
beta <- draws[, "beta"]
gamma <- draws[, "gamma"]

# --- ANNUAL PROJECTIONS (2026-2036) ---
forecast_years <- 2026:2036

pred_matrix <- sapply(forecast_years, function(y) {
```

```
  t <- scale_year(y)
  alpha + beta * t + gamma * t^2
})

projections <- data.frame(
  year = forecast_years,
  pred_mean = colMeans(pred_matrix),
  lower_95 = apply(pred_matrix, 2, quantile, probs = 0.025),
  upper_95 = apply(pred_matrix, 2, quantile, probs = 0.975)
)

#print(projections)

hist_years <- seq(min(train_data$year), max(train_data$year), length.out = 200)
hist_matrix <- sapply(hist_years, function(y) {
  t <- scale_year(y)
  alpha + beta * t + gamma * t^2
})

hist_fit <- data.frame(
  year = hist_years,
  fit_mean = colMeans(hist_matrix),
  lower_95 = apply(hist_matrix, 2, quantile, probs = 0.025),
  upper_95 = apply(hist_matrix, 2, quantile, probs = 0.975)
)

# --- VISUALIZATION ---
forecast_plot <- ggplot() +
  geom_point(
    data = train_data,
    aes(x = year, y = yearly_average, color = "Yearly␣average"),
    size = 1
  ) +
  geom_ribbon(
    data = hist_fit,
    aes(x = year, ymin = lower_95, ymax = upper_95),
    fill = "green4", alpha = 0.15, inherit.aes = FALSE
  ) +
  geom_line(
    data = hist_fit,
    aes(x = year, y = fit_mean, color = "Quadratic␣Fit"),
    linewidth = 0.8
  ) +
  geom_vline(xintercept = 2025, linetype = "dotted", color = "gray40",
      linewidth = 0.5) +
  geom_ribbon(
    data = projections,
    aes(x = year, ymin = lower_95, ymax = upper_95),
    fill = "green4", alpha = 0.25, inherit.aes = FALSE
  ) +
```

```
  geom_line(
    data = projections,
    aes(x = year, y = pred_mean, color = "Projection (2026-2036)"),
    linewidth = 1, linetype = "dashed"
  ) +
  scale_color_manual(
    name = "Data & Models",
    values = c(
      "Yearly average"           = "firebrick",
      "Quadratic Fit"            = "green4",
      "Projection (2026-2036)"   = "green4"
    )
  ) +
  coord_cartesian(ylim = c(310, 450)) +
  scale_x_continuous(breaks = seq(1960, 2040, by = 10)) +
  theme_minimal() +
  labs(
    title = expression("Mauna Loa CO"[2]*" Stan Quadratic Forecast"),
    subtitle = "95% credible bands; yearly averages from imputed data through
        2025",
    x = "Year",
    y = expression(paste("Average CO"[2], " concentration (ppm)"))
  )

ggsave(FORECAST_PLOT_PATH, forecast_plot, width = 10, height = 6, dpi = 300)
#cat("Saved plot to:", FORECAST_PLOT_PATH, "\n")

forecast_plot
```

Listing 26: CO2 forecast with credible bands

```

# Data already loaded from prediction.R in previous chunk
# Display the projections table
kable(projections,
      digits = 2,
      booktabs = TRUE,
      col.names = c("Year", "Mean Prediction (ppm)", "Lower 95% CI", "Upper 95%
          CI"))
```

Listing 27: Ten-year CO2 projections

References